

COMP3320/6464 Week 7, Lecture 1

Rhys Hawkins

Semester 2, 2024

Lecture Plan

This week

- Assignment 1 Wrap-up
- Assignment 2 Preview
- Parallel CPU Motivation
- Pthreads Introduction

Introductions

- New Lecturer
 - Associate Professor Haibo Zhang
 - Visiting from University of Otago
- Starting from next week

Assessment

The total workload for this (6 units) course is around 150 hours (around 12 hours per semester week). For semester 2 2022, the following assessment protocol is proposed:

- **Assignments – 40% of final mark**

- There will be two assignments, each worth 20% of the final course mark (in total 40% of the final course mark).
- These assignments will take the form of programming tasks and/or answering practical and theoretical questions relevant to the course content.

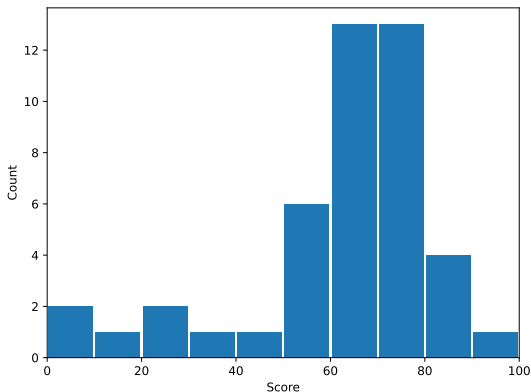
- **Mid-Semester Test – 20% of final mark**

- 1 hour exam held in week 4, worth 20% of your final course mark.
- If you missed the mid-semester exam for a legitimate reason (e.g. medical) you will be awarded zero and your final course make will be derived by scaling your final exam mark to represent 60%.

- **Final Exam – 40% of final mark**

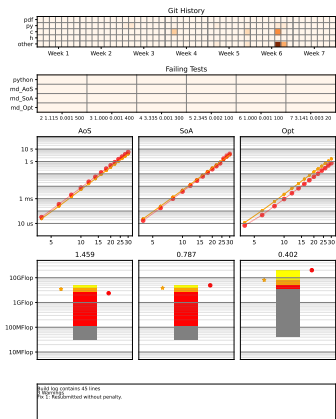
- This will be a 3 hour written exam held at the end of the semester during the normal examination period.
- This exam is worth 40% of the final course mark.

Assignment 1 Marks



- Pretty good distribution.
- Most students with low marks are because there were issues with the submission that will be resolved. Check your email!

Assignment 1 Feedback



- 1 Git commits
- 2 Test results, dark red indicates a failure
- 3 Performance scaling of your codes (orange) with N and against reference solution (red).
- 4 Relative performance to the class and reference solution. Yellow is top 10%, orange top 25%, red top 50%. Number indicates relative performance to reference solution (less than one means slower).
- 5 Notes on build issues.

Contents Covered

An established table of contents for the second part of the course.

- Motivation for Parallel Computers!
- Shared Memory Parallel Programming, Pthreads
- Cloth Simulations (Assignment 2)
- Thread Synchronization
- Shared Memory Parallel Programming with OpenMP (2-3 lectures)
- Shared Memory Computer Architecture: Snooping-Based Cache-Coherence
- Shared Memory Computer Architecture: Directory-Based Cache Coherence
- Parallel Scalability Analysis: Strong and Weak Scaling
- Review of SIMD and SIMD using OpenMP
- Performance Assessment with Roofline CPU
- Review and exam preparation

Strong overlap of part II with COMP4300 — Introductory course to HPC, focused on CPU and SIMD, less content, open to students from fundamental sciences on top of CS.

Textbooks

A good, however not a comprehensive list for the second part of the course:

- Chapters 4, 5, 6, 7 from Introduction to High Performance Computing for Scientists and Engineers (Chapman & Hall/CRC Computational Science) Georg Hager and Gerhard Wellein.
- Chapter 12 from Computer Systems A Programmer's Perspective, Third Edition, Randal E. Bryant and David R. O'Hallaron, Pearson Education Heg USA, ISBN 9781292101767.
- Chapters 2, 3, 4 from Using OpenMP Portable Shared Memory Parallel Programming, Barbara Chapman, Gabriele Jost and Rudd van der Pas, MIT Press, ISBN 9780262533027.
- Chapters 2, 3 from Using OpenMP - The Next Step, Rudd van der Pas, Eric Stotzer, and Christian Terboven, MIT Press, ISBN 9780262534789 (I will provide pdf for these chapters).
- Excerpts (I will provide pdf) from Chapters 5, 6 from Parallel Computer Architecture A Hardware/Software Approach, D. E. Culler, J. P. Singh, and A. Gupta, Morgan Kaufmann Publishers, Inc., ISBN-13: 9781558603431

References

- R. Dennard et al., “Design of Ion- Implanted MOSFETs with Very Small Physical Dimensions,” IEEE J. Solid State Circuits, vol. 9, no. 5, 1974, pp. 256–268.
- M. T. Bohr and I. A. Young, “CMOS Scaling Trends and Beyond,” in IEEE Micro, vol. 37, no. 6, pp. 20-29, November/December 2017, doi: 10.1109/MM.2017.4241347.
- M. B. Taylor, “A Landscape of the New Dark Silicon Design Regime,” in IEEE Micro, vol. 33, no. 5, pp. 8-19, Sept.-Oct. 2013, doi: 10.1109/MM.2013.90.
- The Free Lunch Is Over. A Fundamental Turn Toward Concurrency in Software. Herb Sutter.
<http://www.gotw.ca/publications/concurrency-ddj.htm>

What is parallelism and why do we want to exploit it?

To maximize performance

To maximize performance

- To solve problems as quickly as possible
- Involves efficient cooperation between multiple processing elements operating in parallel

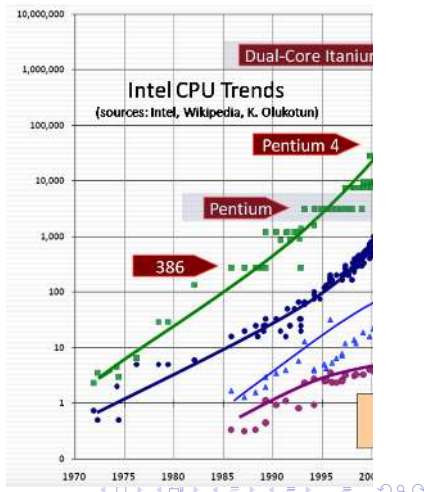
Why?

Why parallel computers?

Until early 2000s – The uniprocessor era

Performance of uniprocessors (single core) increased exponentially (roughly of 40% every 24 months)

- Exponential increase of clock rate
- Increase in Instruction Level Parallelism (ILP)
 - Pipelining: from 5 up to 31 (Prescott) stages
 - Superscalarity: from 1 instruction/clock to 4+ instructions/clock
- Many transistors for additional optimizations
 - Increase in cache sizes
 - Prefetching
 - Sophisticated branch prediction logic



Until early 2000s – The uniprocessor era

Moore's law

- The number of transistors per chip doubles every 24 months while the cost remains constant.
- Does this mean that performance (per chip) doubles every 24 months?

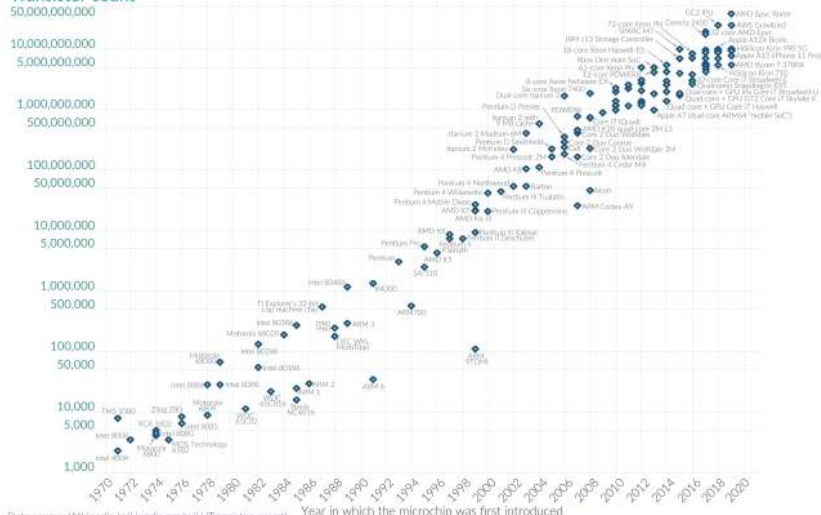
Transistor count

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

Transistor count



Until early 2000s – The uniprocessor era

Dennard Scaling

The power in chips can be modelled as

$$P = QfCV^2$$

where f is frequency, Q the number of transistors, C the capacitance and V the switch voltage

Device or circuit parameter Scaling factor

Device dimension t_{ox} , L , W $1/\kappa$

Doping concentration N_A κ

Voltage V $1/\kappa$

Current I $1/\kappa$

Capacitance $\epsilon A/t$ $1/\kappa$

Delay time/ circuit VC/I $1/\kappa$

Power dissipation/ circuit VI $1/\kappa^2$

Power density VI/A 1

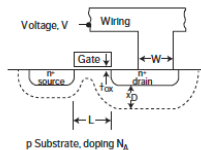


Figure 1. Traditional MOSFET scaling as described by Robert Dennard.

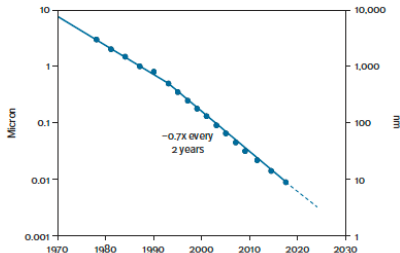


Figure 2. Minimum feature size scaling trend for Intel logic technologies.

Until early 2000s – The uniprocessor era

Dennard Scaling

If we scale feature size down by a factor of $1/k$

$$\frac{P_0}{Q_0} = f_0 C_0 V_0^2 \rightarrow \frac{P_k}{Q_0} = \frac{f_k C_0 V_0^2}{k^3}$$

where $f_k = f_0 k$.

The number of transistors Q that keeps the power consumption constant, while increasing the frequency of a factor of k , is

$$P_k = P_0 \rightarrow Q = k^2 Q_0$$

As long as we keep decreasing feature size exponentially, we can fit k^2 more transistors on the same chip, increase their frequency by k , and use the same power as before!

The end of Dennard scaling (2002-2004)

With feature size below $k < 65\text{nm}$ (currently 14nm)

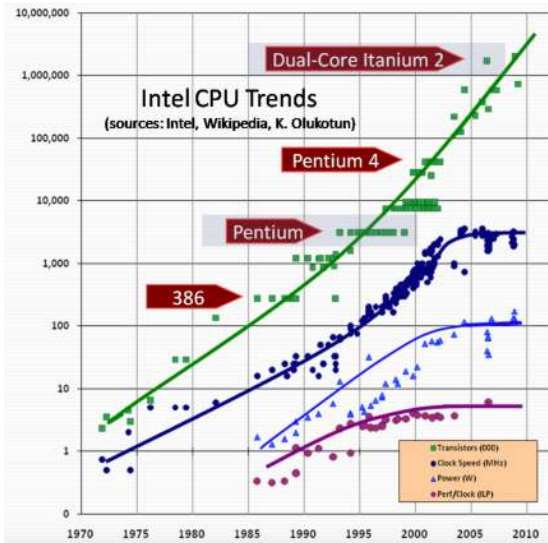
$$P = QfCV^2 + VI_{\text{leakage}}$$

The leakage current grows exponentially with the voltage, as we decrease feature size $1/k$,

$$f = kf_0, Q = k^2Q_0, P_k = k^2P_0$$

To keep the same power envelope, large number of transistors are switched off (**dark silicon effect**), operated at lower frequencies (**dim silicon effect**) or organized in different ways.

The end of the uniprocessor era



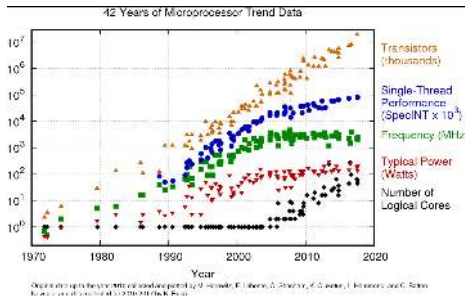
- Clock rate capped (< 4 GHz) by power constraints
- ILP (superscalarity and pipelining) saturated
- Power saturated (limitations in heat removal)
- The free lunch is over

A note on power

Why do we care so much about power?

- shifted from desktop to our smart phone
- democratization of HPC: e.g. ultrascale cloud computing (costs $> 50\%$ power)
- IoT
- physical limitations! Micro-processor are at their thermal limit.
- energy efficiency is the limitation
- power supply limitations

The multicore era



- This is very much different from the golden years of ILP where hardware architects did all the work for us.
- Programmers are now forced to bear the burden of finding and exploiting parallelism.
- This is also an exciting era of opportunities for computational scientists: new algorithms and efficient implementations make a difference on what is achievable in computing.

Assignment 2

- Introduced next week, but it is available now for forking

`https://gitlab.cecs.anu.edu.au/comp3320/2024/comp3320-2024-assignment-2`

- Please get started early, and please ensure timely submission
- Final exam is tentatively scheduled for November 12th, 2:00pm to 5:15pm
- Will be confirmed at the end of September.